# End-to-End MLOps for Heart Disease Classification

**AIMLCZG523 | Assignment 01 | Candidate ID: 2025AE05412**

**Report date:** 21 October 2026 **Repository:** github.com/saktthevel-aiml/mlops

## Executive summary

This report analyses the repository implementation of a reproducible classification workflow using the Cleveland subset of the UCI Heart Disease dataset. The project covers data acquisition and cleaning, exploratory data analysis (EDA), preprocessing and model selection, MLflow experiment tracking, automated tests and GitHub Actions, an HTTP prediction API, Docker packaging, local Kubernetes deployment, and Prometheus/Grafana monitoring.

The committed dataset contains 303 rows, 13 input features and one binary target. Six feature values are missing and retained for median imputation. The training workflow compares Logistic Regression and Random Forest using a stratified 80/20 holdout split and stratified five-fold cross-validation on the training partition. Logistic Regression is selected using mean cross-validation ROC-AUC (0.9001); its held-out ROC-AUC is 0.9578. The complete preprocessing and estimator pipeline is saved for serving.

Repository tests pass (12 passed) and Ruff reports no lint findings in the current environment. Captured evidence shows two successful MLflow runs, a successful GitHub Actions run, successful API checks, two running Minikube replicas, a Prometheus target in the UP state, and a provisioned Grafana dashboard. The Minikube screenshot shows the LoadBalancer external address as pending, so deployment is verified through local port-forwarding rather than a public endpoint.

The project is an academic demonstration, not a clinical prediction system. Before final submission, pin dependencies more tightly, add a Docker build check to CI, verify a fresh clean environment, and include the required short demo video.

## 1. Assignment objective and system scope

The assignment asks for an end-to-end machine-learning solution that can be reproduced, tested, tracked, packaged, deployed, and monitored. It assigns marks to data/EDA, model development, MLflow tracking, packaging, CI and testing, containerization, deployment, monitoring, and documentation. The implementation is evaluated here against those requirements rather than described as a production clinical product.

The system is organized as a batch training path and an online serving path. The training path fetches the public dataset, normalizes its schema, generates EDA plots, compares candidate pipelines, logs each run to MLflow, and writes a selected model artifact. The serving path loads that fitted pipeline into FastAPI, validates incoming JSON, returns a class and model confidence, and exposes operational counters and latency histograms. Docker and Kubernetes configuration package and run the API; Prometheus scrapes the metrics endpoint and Grafana provisions a dashboard.

**Scope boundaries.** The repository contains a local Kubernetes manifest and operational screenshots, not a public-cloud endpoint. CI validates linting, tests, data generation, and training and uploads artifacts; it is not a continuous deployment workflow. Its current YAML does not build a Docker image. These distinctions matter because local proof and cloud release are different acceptance claims.

The implementation has meaningful separation of concerns: `src/data.py` owns acquisition and schema cleaning, `src/modeling.py` owns model construction and training, `src/predict.py` provides command-line inference, and `src/api.py` serves predictions. The notebook supports EDA and interactive explanation, while tests exercise the data, training, prediction, and API layers.

The saved `joblib` object includes preprocessing and the estimator as one pipeline. This avoids a separate serving-time transformation implementation, reducing training/serving skew. The HTTP confidence is the largest class probability; it is not a calibrated probability of an individual's clinical risk.

## 2. Dataset acquisition, schema, and quality

The data loader fetches UCI Machine Learning Repository dataset 45 and selects the Cleveland feature schema. The checked-in CSV has **303 observations and 14 columns**: 13 numeric inputs and `target`. The source target, commonly encoded from 0 through 4, is binarized as 0 for no disease recorded and 1 for a nonzero disease label. This is a dataset-label transformation, not a diagnosis or prospective risk definition.

| Data check | Observed value |
|---|---|
| Rows / columns | 303 / 14 |
| Target 0 | 164 (54.1%) |
| Target 1 | 139 (45.9%) |
| Missing `ca` values | 4 |
| Missing `thal` values | 2 |
| Missing target values | 0 |

The distribution is reasonably balanced for a small educational benchmark, but neither class proportions nor evaluation scores establish general-population performance. The six missing predictor values are converted to numeric missing values and retained. The modeling pipeline imputes them using the median learned from the training data. The target is validated and rows without a usable target would be removed; the current CSV has none.

The 13 features are `age`, `sex`, `cp`, `trestbps`, `chol`, `fbs`, `restecg`, `thalach`, `exang`, `oldpeak`, `slope`, `ca`, and `thal`. The loader checks the expected feature names, normalizes labels to lowercase, coerces values to numeric, and raises an explicit error if required columns or valid target values are missing.

The download path is reproducible through `python -m src.data --output data/heart_disease.csv`. Source: UCI Heart Disease dataset 45. The dataset is small and historical; its sampling, coding conventions, and population representativeness limit conclusions.

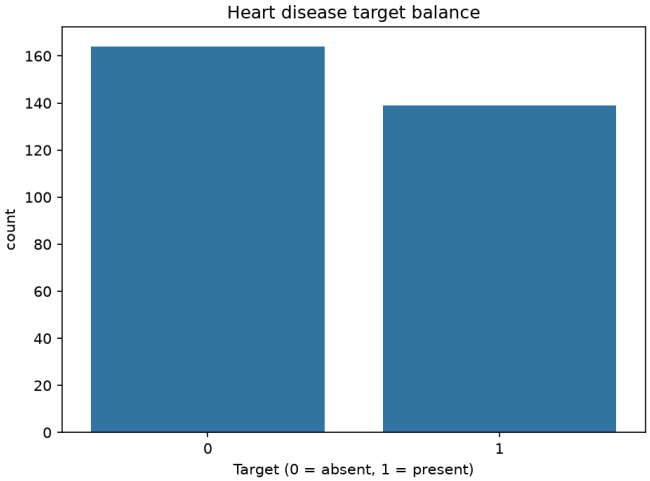



*Figure 1. Class-count plot generated from the repository dataset.*

## 3. Exploratory data analysis

The notebook and training workflow produce a target count plot, feature histograms, and a numeric-correlation heatmap. The plots are generated from the cleaned 303-row dataset and are available under `artifacts/plots/`. Continuous measurements have visibly different ranges and distributions: for example, recorded cholesterol spans 126-564 and maximum heart rate spans 71-202. This motivates scaling for a linear model and makes visual inspection useful for spotting skew and extreme values.

The feature distributions are not interpreted as population estimates. Several fields are coded categories stored numerically, so a histogram or Pearson correlation over their integer codes is descriptive rather than proof of an ordinal clinical effect. In the current data, the largest absolute pairwise target correlations among the numeric fields are `thal` (0.526), `ca` (0.460), `exang` (0.432), `oldpeak` (0.425), and `thalach` (-0.417). These are univariate associations in this dataset only; they do not establish causation or feature importance after adjustment.

The class plot confirms that both labels have adequate representation for stratified splitting and folds, while the histogram panel makes unit/range differences apparent. The heatmap is useful for identifying correlated inputs and possible redundancy; it is not used as a feature-selection rule. No patient-level identifiers are used.

Missingness is concentrated in two columns: four values in `ca` and two in `thal`. The pipeline uses median imputation inside cross-validation, so each fold estimates imputation values from that fold's training partition. This is preferable to calculating imputation statistics on the complete dataset before cross-validation because the latter would leak validation-fold information.

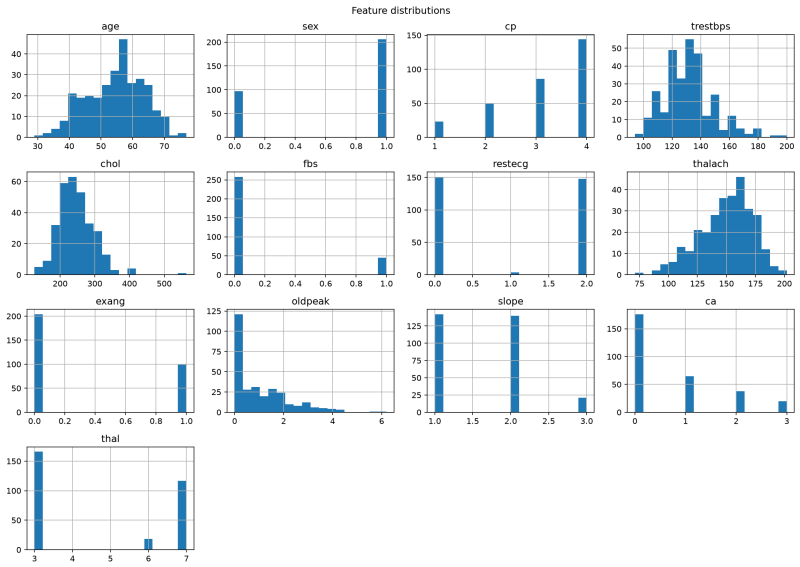



*Figure 2. Numeric input distributions; axes retain different feature units.*

## 4. Modeling method and architecture

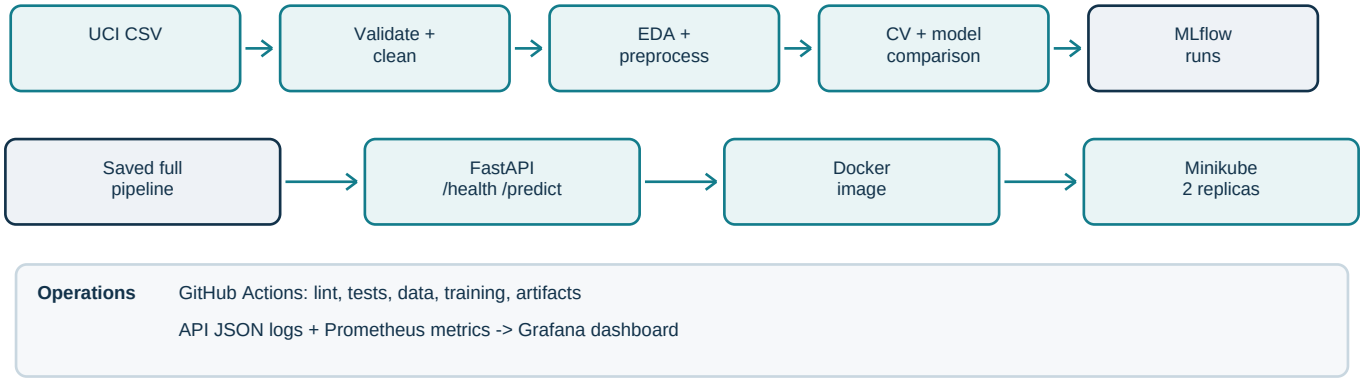
The training code creates a stratified 80/20 train/test split with `random_state=42`. A `StratifiedKFold` with five shuffled folds and the same seed is applied only to the training partition. `GridSearchCV` scores candidates by ROC-AUC and refits the best parameter combination. The held-out test set is evaluated after selection, keeping the model-selection decision separate from the final test evaluation.

Each candidate is a scikit-learn pipeline. A `ColumnTransformer` selects the 13 numeric columns; a median `SimpleImputer` and `StandardScaler` are fitted within the pipeline before classification. The Logistic Regression grid searches `C` over 0.1, 1.0, and 10.0. The Random Forest grid searches 100 or 250 trees and maximum depth of `None`, 4, or 8; class weights are balanced and random state is fixed.

The model is selected by the highest mean cross-validation ROC-AUC, not by test performance. This is important: the Random Forest has higher test accuracy and recall in the current split, but selecting it based on those results would use the holdout test set for tuning. Logistic Regression therefore remains the selection under the stated protocol.

**Architecture.** The diagram below separates offline model lifecycle from online serving and shows monitoring and CI as connected operational layers.

**OFFLINE MODEL LIFECYCLE AND ONLINE SERVING**




The local deployment manifest defines two API replicas, readiness and liveness probes, CPU/memory requests and limits, and a `LoadBalancer` Service. The observed Minikube run uses a local image; a cloud deployment would need a registry image and cluster-specific configuration.

## 5. Evaluation results and model choice

The current `artifacts/metrics.json` records both candidates and their selected hyperparameters. The 20% stratified holdout contains 61 rows. Logistic Regression and Random Forest achieve the same rounded held-out ROC-AUC, while Random Forest has stronger accuracy, precision, and recall on this single split. The pre-declared selection metric is mean cross-validation ROC-AUC.

| Model | Selected parameters | CV ROC-AUC | Test accuracy | Test precision | Test recall | Test ROC-AUC |
|---|---|---|---|---|---|---|
| Logistic Regression | `C=0.1` | **0.9001** | 0.8525 | 0.8065 | 0.8929 | 0.9578 |
| Random Forest | `max_depth=4`, `n_estimators=250` | 0.8903 | **0.8852** | **0.8387** | **0.9286** | 0.9578 |

The selection is Logistic Regression because its mean cross-validation ROC-AUC is higher by approximately 0.0097. Its confusion matrix and ROC curve are generated and logged with the run. The metrics illustrate why a single score is insufficient: the Random Forest's higher test recall may matter in some decision settings, but this dataset and exercise do not define a clinical operating threshold, error cost, calibration requirement, or external validation set.

The measured results are estimates from one split of a small dataset. Cross-validation reduces dependence on one train/validation partition for model selection but does not eliminate sampling uncertainty. No confidence intervals, subgroup fairness analysis, calibration analysis, prospective evaluation, or external cohort validation are claimed.

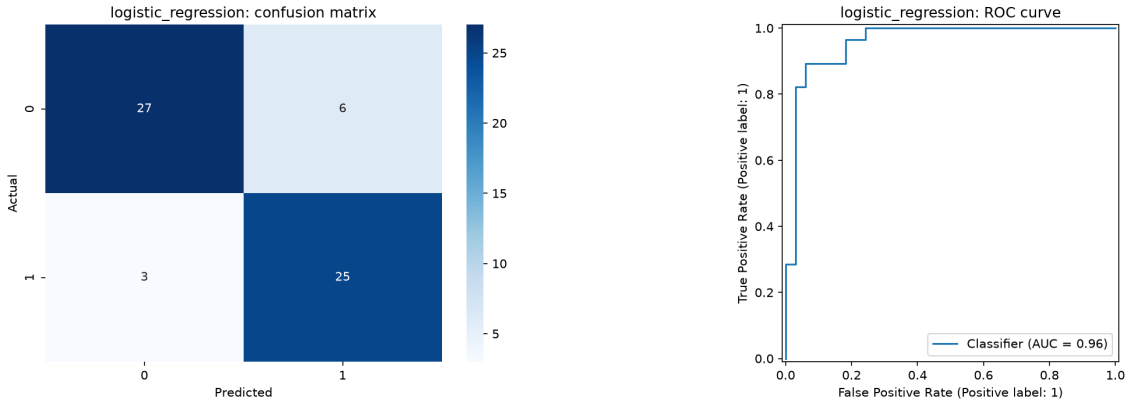



*Figure 3. Held-out evaluation plots for the selected Logistic Regression pipeline.*

## 6. Experiment tracking and reproducibility

The project uses MLflow experiment `uci-heart-disease-classification`. Each candidate run logs the model name, selected search parameters, random seed, fold count, cross-validation ROC-AUC, and held-out accuracy, precision, recall, and ROC-AUC. It also logs a classification report, model signature, input example, fitted model, EDA plots, confusion matrix, and ROC curve.

The repository's current MLflow database is `mlruns/mlflow.db`; the notebook is configured to use a SQLite tracking URI rather than MLflow's maintenance-mode filesystem tracking backend. The current metrics file identifies the two latest recorded candidate run IDs, `f63495b4d4724abd9701c54a49272e58` and `94a3bd6ebe3747229bf43174f9a54896`. The MLflow screenshot shows both completed model runs in the experiment.

The selected inference artifact is `artifacts/model.joblib`. The serialized object is the full preprocessing-and-classification pipeline, so API inference receives the original feature fields and applies the same imputation, scaling, and estimator steps as training. `artifacts/metrics.json` is a compact record of the selected model, metric definition, per-model results, and expected feature order.

The inspected local environment used Python 3.12.12, MLflow 3.17.1, scikit-learn 1.9.1, NumPy 2.5.4, and pandas 2.3.3. These are observations from the environment used for verification, not a lock file. `requirements.txt` currently permits `mlflow>=2.12,<4`, while notebook/model logging uses MLflow 3's `name` parameter. For robust fresh setup, tighten that minimum to MLflow 3 and constrain compatible SQLAlchemy; keep the database-backed tracking URI consistent in scripts, notebook, and UI command.

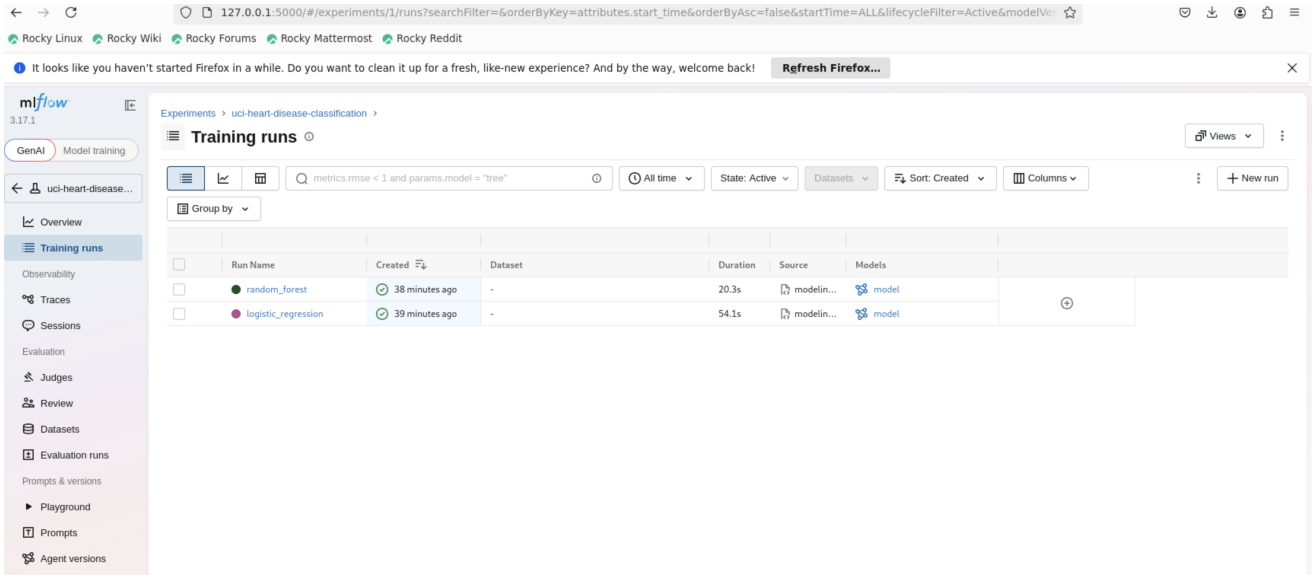



*Figure 4. Captured MLflow experiment with Logistic Regression and Random Forest runs.*

## 7. Automated tests and CI workflow

The test suite covers data cleaning and validation, preprocessing/model candidate setup, training outputs, cross-validation-based selection, command-line prediction logic, and API behavior including valid prediction, invalid input, missing model, and schema/health responses. The current test run completed with **12 passed** and four warnings. Ruff completed with **All checks passed!** The warnings are dependency/API deprecations (including FastAPI's `on_event` lifecycle hook); they did not fail tests but should be addressed during maintenance.

GitHub Actions is defined in `.github/workflows/ci.yml`. On pushes to `main`/`master` and pull requests it checks out code, configures Python 3.12, installs development requirements, runs Ruff and pytest, downloads/generates the dataset, trains both candidates, and uploads artifacts. The included Actions screenshot shows a successful `lint-test-train` job and all listed steps completed.

This is continuous integration, not full continuous delivery. The current workflow does not build or publish the Docker image, deploy to Kubernetes, or run integration tests against a container. The successful run therefore verifies the steps listed above only; the local Docker and Minikube screenshots are separate evidence. A follow-up improvement is to add a Docker build smoke test after the test/training stages and publish only from a protected release workflow.

The test suite uses synthetic data for training tests and changes the working directory to a temporary location, keeping test MLflow output separate from the repository's tracked runs. The data-acquisition and model-training workflow can still require UCI network availability when the CSV is absent.

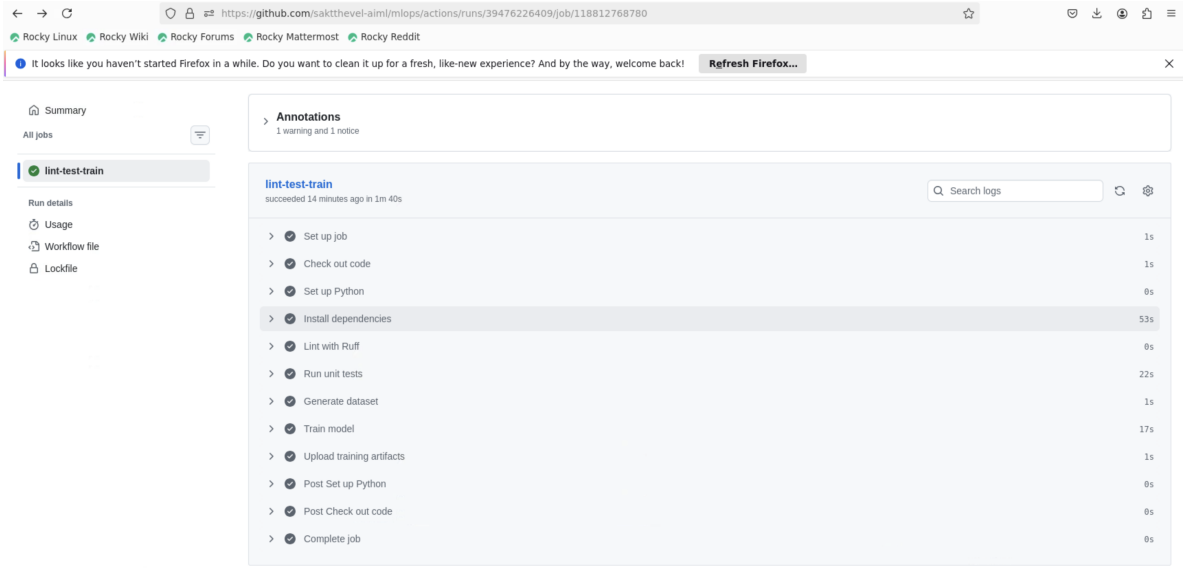



*Figure 5. Successful CI job showing dependency installation, lint, tests, data generation, training, and artifact upload.*

## 8. API contract and container packaging

The FastAPI service loads `artifacts/model.joblib` at startup. `GET /health` reports whether the model loaded; `GET /schema` returns the 13 input features and target labels; `POST /predict` validates feature ranges and rejects unknown fields; `GET /metrics` exposes Prometheus-format metrics. A successful prediction returns the predicted dataset class, class probabilities, maximum probability as confidence, and an explicit educational-use disclaimer. If the model is missing, prediction returns HTTP 503; invalid payloads are rejected by Pydantic validation.

The API increments `prediction_requests_total` by outcome and observes request duration in `prediction_request_duration_seconds`. It emits structured JSON prediction logs. Input constraints are sanity checks for the example schema, not a complete clinical validation policy.

The Dockerfile starts from `python:3.11-slim`, installs runtime requirements, copies source and the model artifact, exposes port 8000, runs as non-root UID 10001, and launches Uvicorn. `docker-compose.yml` maps host port 8000 to the API. The image requires the model to be present at build time, so training or obtaining the intended artifact must precede the image build. The repository screenshots show the API documentation, successful health/prediction calls, and metrics response.

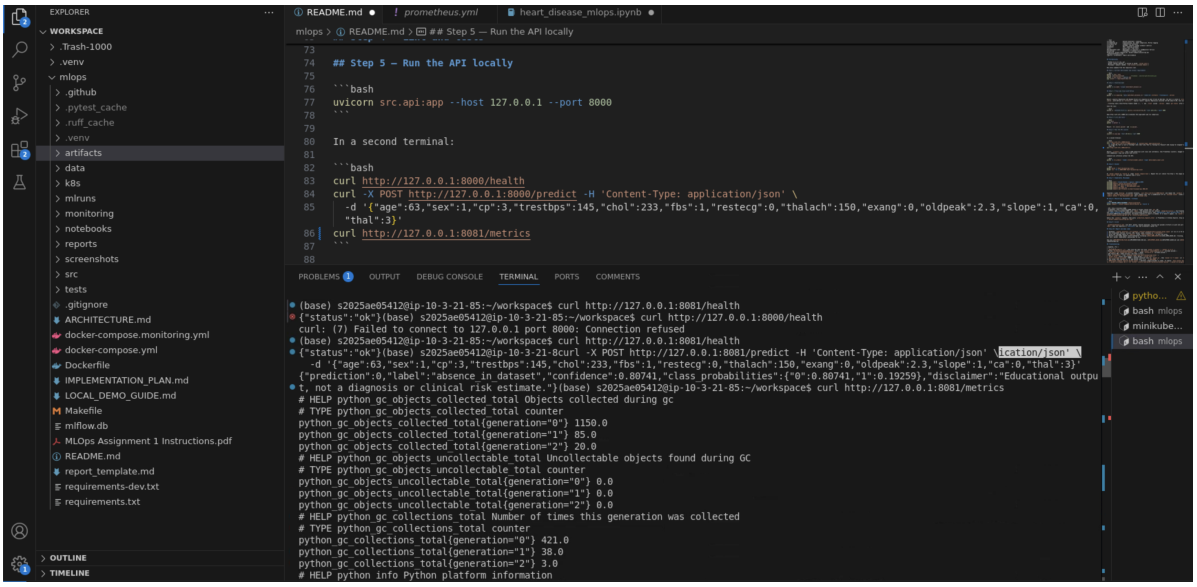



*Figure 6. Captured local API smoke checks; the sample payload is synthetic.*

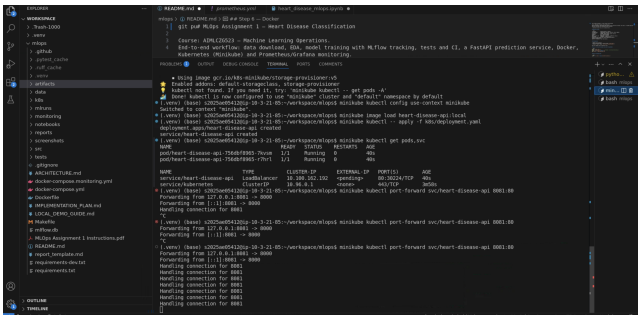
The prediction confidence must not be described as calibrated personal risk. This dataset is not suitable for real-world diagnosis, treatment decisions, or individual clinical advice.

## 9. Local Kubernetes deployment and monitoring

`k8s/deployment.yaml` declares a Deployment with two replicas and a Service of type `LoadBalancer` on port 80 targeting the API's port 8000. Readiness and liveness probes call `/health`; resources are bounded by CPU and memory requests/limits. The screenshot records both pods in the `Running` state and the local service. Its external IP is `<pending>`, so the demonstrated access path is `kubectl port-forward svc/heart-disease-api 8081:80`, not a public endpoint.

The monitoring stack is defined in `docker-compose.monitoring.yml`. Prometheus scrapes `api:8000/metrics` every 10 seconds. Grafana's Prometheus datasource and Heart Disease API dashboard are provisioned from repository configuration. Dashboard panels query prediction counts by status, total predictions, API-up status, p95 latency, and average latency. Captured Prometheus evidence shows the API target as **1/1 UP**; the Grafana screenshot shows the dashboard provisioned and sample counters. Latency charts need enough requests and time-series samples to become informative.

The monitoring is a useful demonstration, not a complete production SRE system. It has no alert rules, persistent metrics storage configuration, authentication hardening, retention policy, or service-level objectives. The compose file's local Grafana password default must be replaced for non-local use; secrets should be injected rather than committed.



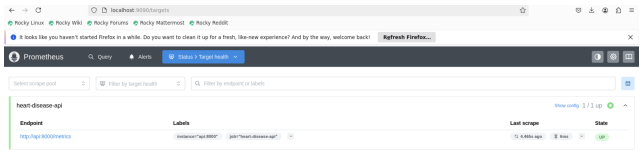


*Figure 7. Minikube replica/port-forward evidence and Prometheus target health.*

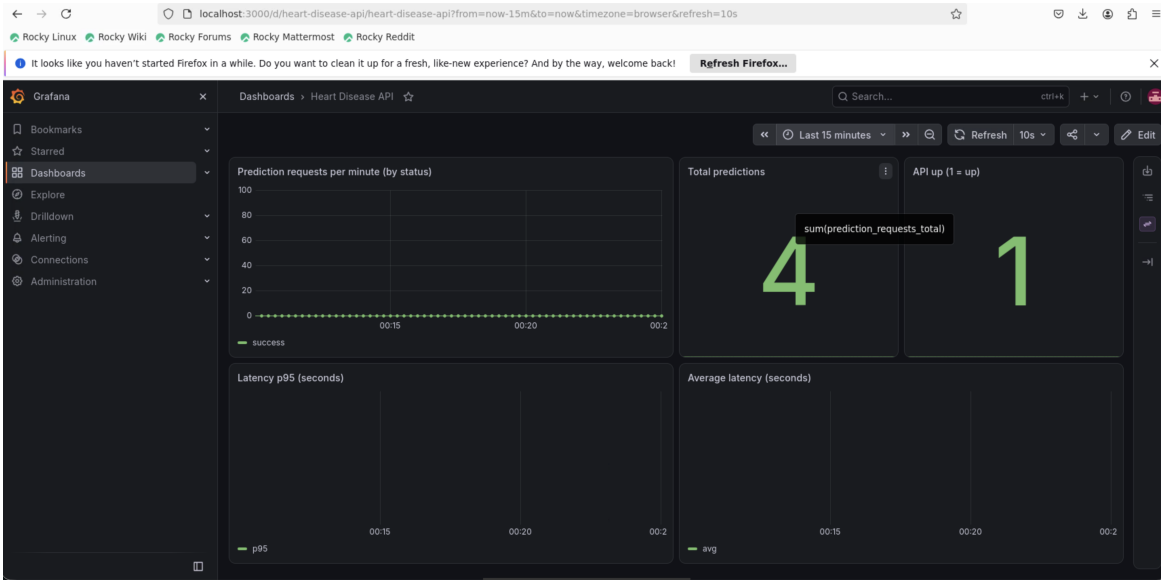



*Figure 8. Provisioned Grafana dashboard with sample request counts and API status.*

## 10. Reproduction, limitations, and completion checklist

Run from the repository root with Python 3.11 or 3.12, Docker, and (for the local cluster) Minikube:

Use a fresh tracking directory for a reproducibility run. The current `resolve_tracking_uri` helper removes entries from its target when it finds legacy directories such as `0`, `1`, or `.trash`; do not point it at an existing MLflow store unless you have backed it up and reviewed the effect.

```
python -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements-dev.txt
python -m src.data --output data/heart_disease.csv
python -m src.modeling --data data/heart_disease.csv --model-dir artifacts --tracking-uri ./mlruns_report
ruff check .
python -m pytest -q
docker compose up --build -d
```

For local Kubernetes, start Minikube, load `heart-disease-api:local`, apply `k8s/deployment.yaml`, and verify the two pods and port-forwarded API. If `kubectl` is absent, `minikube kubectl -- <command>` is available. The repository link is https://github.com/saktthevel-aiml/mlops.

**Verified in this analysis:** 303-row dataset and missingness counts; two model runs and their metrics; 12 passing tests; clean Ruff check; captured successful CI, API, MLflow, Minikube, Prometheus, and Grafana evidence. **Not claimed:** a public cloud deployment, non-pending external IP, clinical suitability, or a recorded demo video.

Before submission: (1) pin MLflow and SQLAlchemy to known-compatible versions; (2) verify the complete setup in a clean environment; (3) add a Docker image build/test to CI if required; (4) record the short end-to-end demo video in `reports/`; (5) verify that screenshots and report are present in the pushed repository; and (6) review and personalize the interpretation in your own words.

**Limitations and responsible use.** The data set is small, historically sourced, and not representative of all populations. The observed metrics are from a single holdout split after cross-validation model selection, with no external validation or calibration. Performance can change across populations and acquisition settings. No output should be used for diagnosis, triage, treatment, or health decisions. This work is an educational MLOps demonstration only.